

Enhanced Similarity Degree Algorithm (ESDA)

Author: Generated for user

Purpose: Detailed algorithm specification for computing an enhanced similarity degree between classes in multiclass imbalanced datasets. The algorithm incorporates preprocessing, feature weighting, subclass clustering, Mahalanobis/kernel distances, exponential similarity conversion, regularization, and normalization.

1. Definitions & Inputs

Inputs:\

- Dataset D with N instances and d attributes (numeric or encoded categorical).\
- Class labels: $C = \{C_1, C_2, \dots, C_K\}$.\
- Parameters: α (similarity scaling), λ (regularization weight), m (max clusters per class), `use_kernel` (True/False), `kernel_sigma` (for RBF kernel).

Outputs:\

- Similarity matrix $S \in \mathbb{R}^{K \times K}$ with values in $[0,1]$, where $S[i,j] = \mu(C_i, C_j)$.

2. Assumptions

- Numeric features are normalized or will be normalized in preprocessing.\

- Categorical features are encoded (one-hot or embedding) or mapped to meaningful numeric representations.\
- Dataset size allows clustering per class (if extremely large, use sampling or approximate clustering).

3. Algorithm Overview

High-level steps:\

1. Preprocessing (cleaning, normalization, encoding).\

2. Feature weighting (compute importance and assign weights w_k).\
 3. Per-class clustering (identify up to m subgroups/subcentroids per class).\
 4. Compute weighted distances between all subgroup centroids using Mahalanobis or kernel distance.\
 5. Convert distances to similarities using exponential decay: $S = \exp(-\alpha \cdot d)$.\
 6. Aggregate subgroup similarities to class-level similarities.\
 7. Apply regularization and normalize similarities to $[0,1]$.
-

Step 1 --- Preprocessing

8. Detailed Procedure

Handle missing values (impute mean/median/mode), normalize numeric features (z-score or min-max), encode categorical features (one-hot, binary, or embeddings).

Step 2 --- Feature Weighting

Train a tree-based model (e.g., RandomForest) or compute SHAP values on a small validation set to obtain feature importance scores $\{f_k\}$. Convert to weights $w_k = f_k / \sum(f_k)$. If feature importance is not available, set $w_k = 1/d$.

Step 3 --- Per-Class Clustering

For each class C_i : apply hierarchical clustering or KMeans to instances of C_i to discover up to m subgroups. Compute subgroup centroids $\{v_i^1, v_i^2, \dots, v_i^{m_i}\}$ and subgroup covariance matrices Σ_i^t . Use silhouette or BIC to choose number of clusters m_i ($\leq m$).

Step 4 --- Distance Computation (weighted Mahalanobis)

For two subgroup centroids v_a^p and v_b^q , compute weighted Mahalanobis distance:

$$d_w(v_a^p, v_b^q) = \sqrt{(v_a^p - v_b^q)^T \cdot W \cdot \Sigma_{pq}^{-1} \cdot W \cdot (v_a^p - v_b^q)}$$
 where $W = \text{diag}(\sqrt{w_1}, \dots, \sqrt{w_d})$ and Σ_{pq} is pooled covariance (e.g., $\Sigma_{pq} = (\Sigma_a^p + \Sigma_b^q)/2 + \epsilon I$).

Step 5 --- Kernel Option (optional)

If `use_kernel=True`, compute RBF kernel similarity instead of Mahalanobis distance:

$$k(v_a^p, v_b^q) = \exp(-\|W(v_a^p - v_b^q)\|^2 / (2 * \text{kernel_sigma}^2))$$

Convert kernel to pseudo-distance via $d = -\log(k)/\alpha$ if needed, or directly use k as similarity between subgroups.

Step 6 --- Convert Distances to Similarity

Use exponential decay: $s_{pq} = \exp(-\alpha \cdot d_w(v_a^p, v_b^q))$. Choose $\alpha = 1/\text{median}(\{d\})$ or tune via cross-validation.

Step 7 --- Aggregate to Class Similarity

Aggregate subgroup-level similarities to class-level similarity. Example aggregation:

$$\mu(C_a, C_b) = (1 / (\sum p_size * \sum q_size)) * \sum_{p=1..m_a} \sum_{q=1..m_b} (n_a^p \cdot n_b^q \cdot s_{pq})$$

where n_a^p is size of subgroup p in class a . This weights subgroup similarities by subgroup sizes.

Step 8 --- Regularization

Apply regularization to avoid extremes:

$\mu_{reg} = \lambda \cdot \mu + (1 - \lambda) \cdot \mu_{baseline}$, where $\mu_{baseline} = \text{mean}(\mu \text{ over all class pairs})$. Choose λ in $[0.7, 1.0]$.

Step 9 --- Normalization

Finally, min-max normalize the matrix to $[0,1]$: $S_{norm}[i,j] = (\mu_{reg}[i,j] - \min) / (\max - \min)$.

Diagonal entries $S[i,i]=1$.

5. Pseudocode

Input: Dataset D , classes C , params α, λ, m , `use_kernel`, `kernel_sigma`

Output: Similarity matrix S ($K \times K$)

\

1. Preprocess D (impute, normalize, encode)\

2. Compute feature importances $\rightarrow$ weights w_k \

3. For each class C_i :\

- Cluster instances into m_i subgroups ($m_i \leq m$)\
- Compute subgroup centroids v_i^p and sizes n_i^p and covariance Σ_i^p \

```

4. For each pair (C_a, C_b):
    μ_sum = 0; weight_sum = 0
    For p in 1..m_a:
        For q in 1..m_b:
            If use_kernel:
                s_pq = exp(-||W(v_a^p - v_b^q)||^2 / (2 * kernel_sigma^2))
            Else:
                compute pooled Σ_pq; d = sqrt((v_a^p - v_b^q)^T W Σ_pq^{-1} W (v_a^p - v_b^q))
                s_pq = exp(-α * d)
            μ_sum += n_a^p * n_b^q * s_pq
            weight_sum += n_a^p * n_b^q
    μ_ab = μ_sum / weight_sum

```

5. Compute $\mu_{\text{baseline}} = \text{mean}(\text{all } \mu_{\text{ab}})$

6. $\mu_{\text{reg_ab}} = \lambda * \mu_{\text{ab}} + (1 - \lambda) * \mu_{\text{baseline}}$

7. Normalize all μ_{reg} to $[0,1] \rightarrow S$

8. Return S

9. Computational Complexity & Notes

- Clustering complexity depends on chosen method (e.g., KMeans $O(t \cdot n \cdot k \cdot d)$).

- Distance computations are $O(K^2 \cdot m^2 \cdot d^3)$ worst-case if inverting covariance matrices; use shared pooled covariances or regularization to reduce cost.
- Use sampling or approximate nearest neighbors for large datasets.
- Tune α via median heuristic: $\alpha = 1/\text{median}(\{d_w\})$.
- Set ϵ (regularization to covariance) small (e.g., $1e-6$) to ensure invertibility.

7. Example (small)

Example: 4 classes, $d=6$ attributes. For each class find 1-2 subgroups, compute centroids, pooled covariances, then compute μ using above steps. This yields a 4×4 similarity matrix used to guide resampling.

8. Implementation Hints

- Libraries: numpy, pandas, scikit-learn (KMeans, AgglomerativeClustering), scipy (linalg), shap (optional).\

- Visualization: seaborn heatmap, scipy dendrogram.\
- Validation: compare with baseline similarity measures (Euclidean- $\rightarrow 1/(1+d)$, cosine) and evaluate impact on resampling/classification.

9. References & Further Reading

- Janicka, Lango, Stefanowski --- Using Information on Class Interrelations to Improve Classification of Multiclass Imbalanced Data (SOUP)\

- Papers on Mahalanobis distance, kernel methods, SHAP values, and clustering for class decomposition.